UNIT-II
CLASSIFICATION & REGRESSION

Classification:

- Classification is a process of categorizing data or objects into predefined classes or categories based on their features or attributes.
- Classification algorithm is a Supervised Learning technique it is used to predict a categorical output. For example, a classification algorithm could be used to predict whether an email is spam or not, Yes or No, 0 or 1, cat or dog, etc.,
- Unlike regression, the output variable of Classification is a category, not a value, such as "Green or Blue", "fruit or animal", etc. Since the Classification algorithm is a Supervised learning technique, hence it takes labeled input data, which means it contains input with the corresponding output.
- The main objective of classification machine learning is to build a model that can accurately assign a label or category to a new observation based on its features.
- For example, a classification model might be trained on a dataset of images labeled as either dogs or cats and then used to predict the class of new, unseen images of dogs or cats based on their features such as color, texture, and shape.

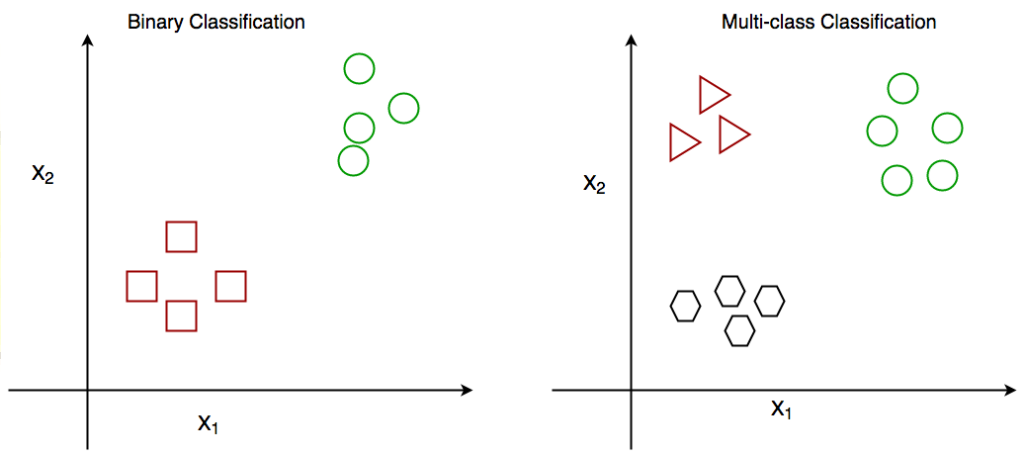
Classification Types

There are two main classification types in machine learning:

1. ***Binary Classification***
   - In binary classification, the goal is to classify the input into one of two classes or categories or outcomes.

     Example: YES or NO, MALE or FEMALE, SPAM or NOT SPAM, CAT or DOG, etc.
2. ***Multiclass Classification***
   - In multi-class classification, the goal is to classify the input into one of several classes or categories.

     Example: Classifications of types of crops, Classification of types of music, Classification among different species of flowers.



3. ***Multi-Label Classification***
   - In, Multi-label Classification the goal is to predict which of several labels a new data point belongs to. This is different from multiclass classification, where each data point can only belong to one class.

     For example, a multi-label classification algorithm could be used to classify images of animals as belonging to one or more of the categories cat, dog, bird, or fish.

4. ***Imbalanced Classification***
   - In, Imbalanced Classification the goal is to predict whether a new data point belongs to a minority class, even though there are many more examples of the majority class.
     For example, a medical diagnosis algorithm could be used to predict whether a patient has a rare disease, even though there are many more patients with common diseases.

Types of ML Classification Algorithms:

There are various types of classification algorithms. Some of them are:

1. ***Linear Models:***
   - Linear models create a linear decision boundary between classes. They are simple and computationally efficient. Some of the linear classification models are as follows:
     - Logistic Regression
     - Support Vector Machines (having kernel = 'linear')
     - Single-layer Perceptron
     - Stochastic Gradient Descent (SGD) Classifier
2. ***Non-linear Models:***
   - Non-linear models create a non-linear decision boundary between classes. They can capture more complex relationships between the input features and the target variable. Some of the non-linear classification models are as follows:
     - K-Nearest Neighbours
     - Kernel SVM
     - Naïve Bayes
     - Decision Tree Classification
     - Random Forest Classification
     - Multi-layer Artificial Neural Networks

Binary Classification:

- In machine learning, binary classification is a supervised learning algorithm.
- In binary classification, the goal is to classify the input into one of two classes or categories or outcomes.
  Example: YES or NO, MALE or FEMALE, SPAM or NOT SPAM, CAT or DOG, etc.

Application	Observation	0	1
Medical Diagnosis	Patient	Healthy	Diseased
Email Analysis	Email	Not Spam	Spam
Financial Data Analysis	Transaction	Not Fraud	Fraud
Marketing	Website visitor	Won't Buy	Will Buy
Image Classification	Image	Hotdog	Not Hotdog

Evaluation of binary classifiers

Consider, in a medical diagnosis, a binary classifier for a specific disease could take a patient's symptoms as input features and predict whether the patient is healthy or has the disease. The possible outcomes of the diagnosis are ***positive*** and ***negative***.

- If the model successfully predicts the patients as positive, this case is called True Positive (TP).
- If the model successfully predicts patients as negative, this is called True Negative (TN).

- The binary classifier may misdiagnose some patients as well.
- If a diseased patient is classified as healthy by a negative test result, this error is called False Negative (FN).
- Similarly, If a healthy patient is classified as diseased by a positive test result, this error is called False Positive(FP).

We can evaluate a binary classifier based on the following parameters:

- True Positive (TP): The patient is diseased and the model predicts "diseased"
- False Positive (FP): The patient is healthy but the model predicts "diseased"
- True Negative (TN): The patient is healthy and the model predicts "healthy"
- False Negative (FN): The patient is diseased and the model predicts "healthy"

After obtaining these values, we can compute the accuracy score of the binary classifier as follows:

$$\text{accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

The following is a confusion matrix, which represents the above parameters:

		PREDICTED	
		Positive	Negative
ACTUAL	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

In machine learning, many methods utilize binary classification. The most common are:

- Logistic Regression
- Support Vector Machines (SVM)
- Decision Trees
- Naive Bayes
- Nearest Neighbor
- Neural Networks

Assessing Classification performance

- In machine learning, classification is the process of categorizing a given set of data into different categories.
- An important element of any machine learning workflow is the evaluation of the performance of the model. This is the process where we use the trained model to make predictions on previously unseen, labeled data.
- In the case of classification, we then evaluate how many of these predictions the model got right.
- In real-world classification problems, it is usually impossible for a model to be 100% correct. When evaluating a model it is, therefore, useful to know, not only how wrong the model was, but in which way the model was wrong.

"All models are wrong, but some are useful", George Box

- Evaluating a classification model is an important step in machine learning, as it helps to assess the performance and generalization ability of the model on new, unseen data.
- There are several metrics and techniques that can be used to evaluate a classification model, depending on the specific problem and requirements. Here are some commonly used evaluation

metrics:

1. Accuracy
2. Logarithmic Loss (Log Loss or Cross-Entropy Loss)
3. Confusion matrix
4. Precision
5. Recall or Sensitivity
6. F1-Score
7. Specificity
8. Area Under the Receiver Operating Characteristic Curve (AUC-ROC)

1. Accuracy:

- Classification accuracy is the proportion of correctly classified instances over the total number of instances in the test set.
- It is a simple and intuitive metric but can be misleading in imbalanced datasets where the majority class dominates the accuracy score.
- An accuracy score will give a value between 0 and 1; a value of 1 would indicate a perfect model.

$$\text{Accuracy} = \frac{\text{No. of correct predictions}}{\text{Total number of input samples}}$$

- It works great if there are an equal number of samples for each class.
- It is good to use the Accuracy metric when the target variable classes in data are approximately balanced. For example, if 60% of classes in a fruit image dataset are of Apple, 40% are Mango. In this case, if the model is asked to predict whether the image is of Apple or Mango, it will give a prediction with 97% of accuracy.
- It is recommended not to use the Accuracy measure when the target variable majorly belongs to one class. For example, Suppose there is a model for a disease prediction in which, out of 100 people, only five people have a disease, and 95 people don't have one. In this case, if our model predicts every person with no disease (which means a bad prediction), the Accuracy measure will be 95%, which is not correct.
- Classification accuracy is good but it gives a False Positive sense of achieving high accuracy. The problem arises due to the possibility of misclassification of minor class samples being very high.

2. Logarithmic Loss (Log Loss or Cross-Entropy Loss):

- It is also called Logistic regression loss or cross-entropy loss.
- It is basically defined as probability estimates and measures the performance of a classification model where the input is a probability value between 0 and 1.
- Its basic working idea is based on penalizing the false (False Positive) classification.
- It usually works well with multi-class classification.
- Working on Log loss, the classifier should assign a probability for each and every class of all the samples.
- If there are N samples belonging to the M class, then we calculate the Log loss in this way:

$$LogarithmicLoss = -1 \frac{}{N \sum_{i=1}^{N} \sum_{j=1}^{M} y_{ij} * \log(p_{ij})}$$

where,

- y_{ij} indicate whether sample i belongs to class j.
- p_{ij} – The probability of sample i belongs to class j.
- The range of log loss is [0,?). When the log loss is near 0 it indicates high accuracy and when

away from zero then, it indicates lower accuracy.

- Therefore, minimizing log loss gives you higher accuracy for the classifier.

3. Confusion matrix:

- A confusion matrix is a matrix that summarizes the performance of a machine learning model on a set of test data.
- It is a means of displaying the number of accurate and inaccurate instances based on the model's predictions.
- It is often used to measure the performance of classification models, which aim to predict a categorical label for each input instance.
- The matrix displays the number of instances produced by the model on the test data.
  - ✓ True positives (TP): occur when the model accurately predicts a positive data point.
  - ✓ True negatives (TN): occur when the model accurately predicts a negative data point.
  - ✓ True positives (FP): occur when the model predicts a positive data point incorrectly.
  - ✓ False negatives (FN): occur when the model mispredicts a negative data point.
- It offers a thorough analysis of true positive, true negative, false positive, and false negative predictions, facilitating a more profound comprehension of a model's recall, accuracy, precision, and overall effectiveness in class distinction.
- When there is an uneven class distribution in a dataset, this matrix is especially helpful in evaluating a model's performance beyond basic accuracy metrics.

Confusion Matrix For binary classification

- Samples of that classification belong to either Yes or No. So, we build our classifier which will predict the class for the new input sample. After that, we tested our model with 165 samples, and we get the following result.

n = 165	**Predicted: Yes**	**Predicted: No**
Actual: Yes	100 (TP)	5 (FP)
Actual: No	10 (FN)	50 (TN)

There are 4 terms are:

- ✓ True Positives: It is the case where we predicted Yes and the real output was also yes.
- ✓ True Negatives: It is the case where we predicted No and the real output was also No.
- ✓ False Positives: It is the case where we predicted Yes but it was actually No.
- ✓ False Negatives: It is the case where we predicted No but it was actually Yes.

- The accuracy of the matrix is always calculated by taking average values present in the main diagonal i.e.

Accuracy = (TP+TN)/Total Sample

= (100+50)/165 = 0.91

4. Precision:

- Precision measures the proportion of true positives over the total number of predicted positives.
- It is calculated as the number of true positive predictions to the total positive predictions (True Positive and False Positive).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Precision = 100/(100+5) = 0.952

5. Recall or Sensitivity:

- It is also similar to the Precision metric; however, it aims to calculate the proportion of actual

positive that was identified incorrectly.

- It is the ratio of the number of true positives (TP) to the total number of actual positives, either correctly predicted as positive or incorrectly predicted as negative (true Positive and false negative).

$$\text{Recall} = \frac{TP}{TP+FN}$$

Recall = 100/(100+10) = 0.909

- Precision and Recall metrics are useful in scenarios where one class is more important than the other, or when there is a trade-off between false positives and false negatives.

***Note*:-** We use precision when we want to minimize false positives, crucial in scenarios like spam email detection where misclassifying a non-spam message as spam is costly. And we use recall when minimizing false negatives is essential, as in medical diagnoses, where identifying all actual positive cases is critical, even if it results in some false positives.

6. F1-Score:

- F-score or F1 Score is a metric to evaluate a binary classification model on the basis of predictions that are made for the positive class.
- It is calculated with the help of Precision and Recall. It is a type of single score that represents both Precision and Recall.
- The harmonic mean of precision and recall, calculated as 2 x (precision x recall) / (precision + recall). Its range is [0, 1]. The more the F1 score better will be performance.
- It is a useful metric for imbalanced datasets where both precision and recall are important.
- F1-score is used to evaluate the overall performance of a classification model.

$$\text{F1-Score} = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

F1-Score = 2*(0.952*0.909)/(0.952+0.909) = 0.93

- We balance precision and recall with the F1-score when a trade-off between minimizing false positives and false negatives is necessary, such as in information retrieval systems.

7. Specificity:

- Specificity is another important metric in the evaluation of classification models, particularly in binary classification.
- It measures the ability of a model to correctly identify negative instances. Specificity is also known as the True Negative Rate.

$$\text{Specificity} = \frac{TN}{TN+FP}$$

Specificity = 10/(10+5) = 0.667

8. Area Under the Receiver Operating Characteristic Curve (AUC-ROC):

- If we need to visualize the performance of the classification model on charts; then, we can use the AUC-ROC curve.
- ROC represents a graph to show the performance of a classification model at different threshold levels.
- The Receiver Operating Characteristic (ROC) curve is a plot of the true positive rate (recall) against the false positive rate (1-specificity) for different threshold values of the classifier's decision function.
- AUC calculates the two-dimensional area under the entire ROC curve.
- The Area Under the Curve (AUC) measures the overall performance of the classifier, with

values ranging from 0.5 (random guessing) to 1 (perfect classification).

True positive rate:

Also known as sensitivity, True Positive Rate is considered as a portion of positive data points that are correctly considered as positive, with respect to all data points that are positive.

$$\text{TPR} = \frac{\text{TP}}{\text{TP+FN}}$$

True Negative Rate:

Also known as specificity, False Negative Rate is considered as a portion of negative data points that are correctly considered as negative, with respect to all data points that are negatives.

$$\text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

False-positive Rate:

False Negative Rate is considered as a portion of negative data points that are mistakenly considered as negative, with respect to all data points that are negative.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

- False Positive Rate and True Positive Rate both have values in the range [0, 1].
- Greater the value of AUC better the performance of the model.

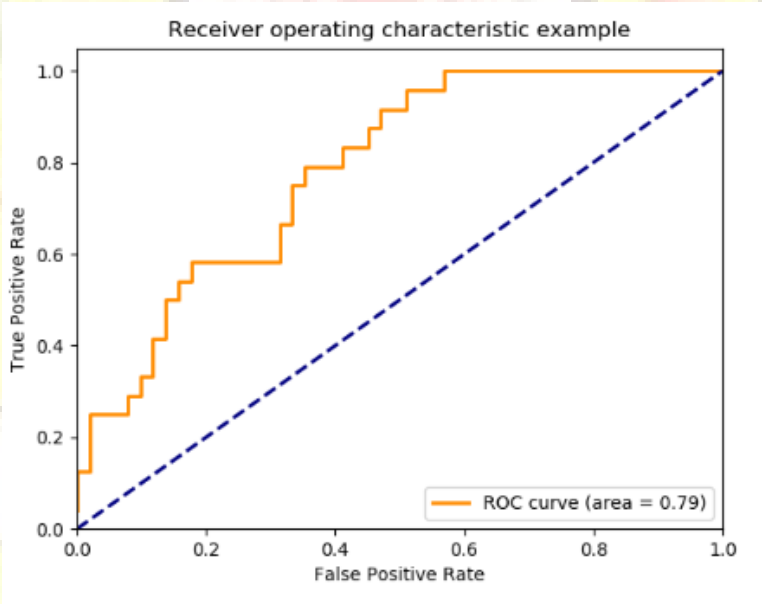


Fig:- ROC Curve for Evaluation of Classification Models

Class probability Estimation

A class probability estimator – or probability estimator in short – is a scoring classifier that outputs probability vectors over classes, i.e., a mapping $\hat{\mathbf{p}} : \mathscr{X} \to [0,1]^k$.

We write $\hat{\mathbf{p}}(x) = (\hat{p}_1(x), \ldots, \hat{p}_k(x))$, where $\hat{p}_i(x)$ is the probability assigned to class C_i for instance x, and $\sum_{i=1}^{k} \hat{p}_i(x) = 1$.

If we have only two classes, the probability associated with one class is 1 minus the probability of the other class; in that case, $\hat{p}(x)$ we use to denote the estimated probability of the positive class for instance x. As with scoring classifiers, we usually do not have direct access to the true probabilities $p_i(x)$.

One way to understand the probabilities $\hat{p}_i(x)$ is as estimates of the probability

$$P_{\mathscr{C}}(c(x') = C_i | x' \sim x),$$

Where, $x^1 \sim x$ stands for "x^1 is similar to x". In other words, how frequent are instances of this class among instances similar to x? The intuition is that the more (or less) frequent they are, the more (or less) confident we should be in our belief that x belongs to that class as well.

Here, we mean with similarity in this context will depend on the models we are considering – we will illustrate it here by means of a few two-class examples.

First, assume a situation in which any two instances are similar to each other. We then have

$$P_{\mathscr{C}}(c(x') = \oplus | x' \sim x) = P_{\mathscr{C}}(c(x') = \oplus)$$

which is simply estimated by the proportion *pos* of positives in our data set. In this scenario we predict $\hat{p}(x) = pos$ regardless of whether we know anything about x's true class.

At the other extreme, consider a situation in which no two instances are similar unless they are the same, i.e., $x^1 \sim x$ if $x^1 = x$, and $x' \neq x$ otherwise.

Calibration Curves:

- Calibration curves are used to evaluate how well-calibrated a classifier is, specifically in terms of the probabilities it predicts for each class label.
- When building a classification model, we often want to predict not only the class labels but also the probabilities of an instance belonging to each class.
- For example, consider a fruit classification model with three class labels: apples, oranges, and bananas. We'd like to know the probabilities of a fruit being an apple, an orange, or a banana.
- Calibration curves help us understand how "sure" a model is while predicting a class label and interpret its decisiveness.
- The x-axis of a calibration curve represents the average predicted probability in each bin.
- The y-axis represents the ratio of positive predictions (i.e., the proportion of positive outcomes).
- The ideal calibrated model's curve is a straight line from (0, 0) to (1, 1).
- If a model's curve closely follows this ideal line, it is well-calibrated.

Mean Squared Error (MSE):

- MSE is a common metric used to evaluate the performance of regression models.
- It quantifies the average squared difference between the predicted values and the actual (observed) values i.e.,

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

where:

- ✓ N - is the total number of data points.
- ✓ y_i - represents the actual value for the ith data point.
- ✓ y_i^1 - represents the predicted value for the ith data point.

- A lower MSE indicates better model performance because it means the predicted values are closer to the actual values.
- Squaring the errors ensures that both positive and negative deviations contribute equally to the overall error.
- However, MSE is sensitive to outliers, as large errors are magnified due to squaring.

Brier Score:

- The Brier score is an evaluation metric used to assess the goodness of predicted probability scores.
- It is similar to the mean squared error (MSE), but specifically applied to prediction probability scores that range between 0 and 1.
- Brier score is commonly used in classification tasks where outcomes are binary (e.g., win/lose, spam/ham, click/no-click).

- Unlike log loss, the Brier score is gentler in penalizing inaccurate predictions.

$$\text{Brier Score} = \frac{1}{N}\sum_{i=1}^{N}(f_t - o_t)^2$$

Here:

- ✓ f_t - represents the predicted value (probability) for the (i)-th observation.
- ✓ o_t - represents the actual outcome (0 or 1) for the (i)-th observation.
- ✓ N - is the total number of observations.

- The Brier score calculates the mean squared error between predicted probabilities and observed values (actuals).
- Its value always falls between 0.0 and 1.0, where a perfect model has a score of 0.0, and the worst model has a score of 1.0.
- Lower Brier scores indicate better model performance.

Reliability Diagrams:

- Reliability diagrams visualize the relationship between predicted probabilities and observed frequencies.
- They help identify over-confidence or under-confidence in model predictions.
- Ideally, the points on the reliability diagram should lie close to the diagonal line.

Logarithmic Loss (Log Loss):

- Log loss evaluates the quality of predicted probabilities for classification tasks.
- It penalizes incorrect predictions more severely.
- Lower log loss values indicate better-calibrated models.

Assessing class probability Estimates:

Have you ever wondered why Logistic Regression is used for classification problem but still has "Regression" in it.

Now we see,

- Why regression term is there in Logistic Regression?
- How it can be converted to class probabilities?

For Classification problems in machine learning we often want to know how likely the instance belongs to the class rather than which class it will belong to. So in many cases we would like to use the estimated class probability for decision making.

Consider a scenario where we have to detect a credit fraud. The manager of fraud control department wants to know not only who are likely to be fraud but also the cases where credit risk is at stake i.e., accounts where companies' monetary loss is expected to be highest.

Here, we must know the class probability of fraud for that particular case.

What exactly is an accurate estimate of class membership probability is a subject of debate beyond the scope that we see here?

Roughly, we would like

i. the probability estimates to be well calibrated, meaning that if you take 100 cases whose class membership probability is estimated to be 0.2, then about 20 of them will actually belong to the class.

ii. the probability estimates to be discriminative. Meaning that they should give different probability estimates for different examples. Say 0.5 class probability indicates that 50% of population is fraudulent. Which is base rate thus we need discrimination to get some higher/lower class probability boundary for estimation?

Understanding what is the difficulty in using Linear Model for predicting class probability?

Assume, ***f(x)*** is our linear function. *x* is an instance being further from the separating boundary intuitively ought to lead to a higher probability of being in one class or the other. Thus ***f(x)*** gives us the distance from the separating boundary. As we know that linear regression can take values from -infinity to +infinity. But our class probabilities range from 0 to 1.

One useful notion of likelihood of an event is odds.

The odd of an event is the ratio of the event occurring to the probability of the event not occurring. Odds range from 0 to +infinity. So we cannot map our linear distribution from 0 to +infinity using odds function.

But, Wait! Since any number which ranges from 0 to +infinity its log value will range from -infinity to +infinity. So, let's compare it i.e., log-odds with our linear model.

Assume we have class c instance to be predicted from linear model then log-odds of it will be

$$\log[\frac{P(c))}{1-P(c)}] = f(x) = w0 + w1 * x1 + w2 * x2$$

Eqn:- Log-odds with linear model also called as Logit Function

Above, *w0, w1, w2, ...* are weights given by our linear model and *x1, x2, x3,...* are features of dataset. ***P(c)*** is the probability of instance that how much it is a credit fraud and ***1-P(c)*** is the probability of instance that how much it is not being a credit fraud.

Now, we often want probability of class c i.e ***P(c)*** as our predicted class probability and don't want log odds for function. We can solve for ***P(c)***.

Thus we take exponential of both the sides w.r.t ***e***.

$${}_e log[\frac{P(c)}{1-P(c)}] =_e f(x)$$

Taking exponential of both sides w.r.t *e*

On left side we can cancel out ***e*** and ***log*** for power term to be the value.

$$\frac{P(c)}{1-P(c)} = e^{f(x)}$$

Solving for ***P(c)***,

$$P(c) = (1-P(c)) * e^{f(x)} \quad => \quad P(c) = e^{f(x)} - P(c) * e^{f(x)}$$

$$P(c) + P(c) * e^{f(x)} = e^{f(x)}$$ Taking P(c) common

$$P(c) * (1 + e^{f(x)}) = e^{f(x)}$$

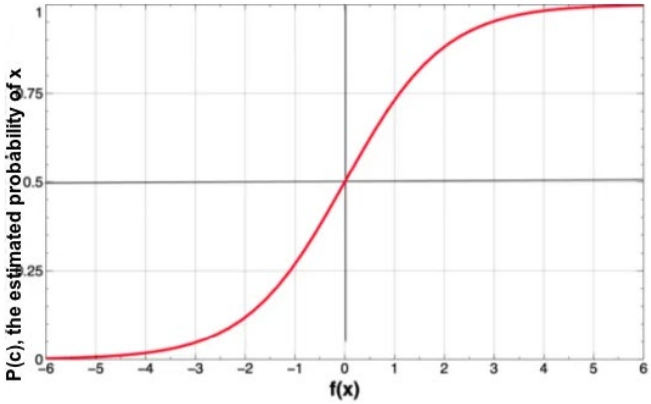
$$P(c) = \frac{e^{f(x)}}{1+e^{f(x)}}$$

Dividing, numerator and denominator on right side by $e^{f(x)}$, we get

$$P(c) = \frac{\frac{e^{f(x)}}{e^{f(x)}}}{\frac{1}{e^{f(x)}} + \frac{e^{f(x)}}{e^{f(x)}}} \quad => \quad P(c) = \frac{1}{\frac{1}{e^{f(x)}} + 1} \quad => \quad P(c) = \frac{1}{1+e^{-f(x))}}$$

Hence, this is the required ***Sigmoid function***.

In above equation, if we plot the graph using values for x and w we get the curve something like



Above curve is called the "Sigmoid Curve" due to its S shape which squeezes the probabilities into their respective correct range (between zero and one).

The sigmoid curve suggests that the values near the boundary are uncertain of class. And as you move away from boundary the uncertainty decreases and thus membership of class becomes certain.

This leads us to the standard objective function for fitting a logistic regression. Thus the "likelihood" of given the example belong to correct class can be given by,

$$g(x, w) = \begin{cases} \text{if x is of class c then } P(c) = 1 \\ \text{if x is not of class c then } 1 - P(c) = 0 \end{cases}$$

Here, the function ***g*** gives the model's estimated probability of seeing x's actual class given x's features.

The maximum likelihood model "on average" gives the highest probabilities to the positive examples and the lowest probabilities to the negative examples.

How do you calculate probability in machine learning?

P(A ∩ B) = Probability of occurring events A and B both. Conditional Probability: It is given by the Probability of event A given that event B occurred. Similarly, ***P(B|A) = P(A ∩ B)/ P(A)***.

Multiclass Classification:

- Multiclass classification is a popular problem in supervised machine learning refers to classification tasks with more than two classes are very common.
- Multiclass classification aims to assign each instance to one of several possible classes based on its features or attributes.
- Given a dataset of m training examples, each of which contains information in the form of various features and a label. Each label corresponds to a class, to which the training example belongs. In multiclass classification, we have a finite set of classes. Each training example also has n features.
- For instance, once a patient has been diagnosed as suffering from a rheumatic disease, the doctor will want to classify him or her further into one of several variants and Imagine labeling images of different fruits (such as oranges, apples, and pears).
- Imagine now that we want to construct a multi-class classifier, but we only have the ability to train two-class models – say linear classifiers.
- The general way of addressing a k-class learning problem with binary classifiers is to
  - (i). break the problem up into *l* binary learning problems;
  - (ii). train *l* binary classifiers on two-class versions of the original data; and
  - (iii). combine the predictions from these *l* classifiers into a single k-class prediction.
- The most common ways to do the first and third step is ***one-versus-one*** or ***one-versus-rest***, but

the use of code matrices gives the opportunity of implementing other schemes.

- Various algorithms can be used for multiclass classification, including:
  1) ***Naïve Bayes:*** A probabilistic model based on conditional probability and Bayes' theorem. It assumes that features are independent, although this assumption is often not true in practice.
  2) ***Decision Trees:*** Tree-based models that recursively split data based on features to create decision rules.
  3) ***Support Vector Machines (SVM):*** A powerful algorithm that finds the best hyperplane to separate classes when the feature vector is high dimensional.
  4) ***Random Forest Classifier:*** An ensemble method combining multiple decision trees.
  5) ***K-Nearest Neighbors (KNN):*** A non-parametric algorithm that classifies based on the majority class among its neighbors using distance metric (Ex. Euclidian distance).
  6) ***Logistic Regression:*** A linear model used for binary and multiclass classification.
- Handling imbalanced data, selecting appropriate features, and choosing the right algorithm are common challenges in multiclass classification.
- Metrics like accuracy, precision, recall, and F1-score are used to evaluate multiclass classifiers.

SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
6.8	3.2	5.9	2.3	Iris-virginica
6.9	3.1	5.1	2.3	Iris-virginica
4.9	3.0	1.4	0.2	Iris-setosa
5.6	3.0	4.5	1.5	Iris-versicolor
4.8	3.1	1.6	0.2	Iris-setosa
5.8	2.8	5.1	2.4	Iris-virginica
7.2	3.6	6.1	2.5	Iris-virginica
5.1	3.5	1.4	0.3	Iris-setosa
4.7	3.2	1.6	0.2	Iris-setosa
6.6	3.0	4.4	1.4	Iris-versicolor

Fig.:- Iris dataset with three categories.

Multiclass Vs Multi-label

- Note, multiclass and multi-label classification are very different and cannot be used interchangeably.
- Multi-label refers to a data point that may belong to more than one class.
- Whereas, in multiclass or binary classification, your data point can belong to only a single class.
- For example, you wish to watch a movie with your friends but you have a different choice of genres that you all enjoy. Some of your friends like comedy and others are more into action and thrill. Therefore, you search for a movie that fulfills both the requirements and here, your movie is supposed to have multiple labels.
- Another example of the multi-label dataset could be protein classification in the human body.

Regression:

- It is a supervised machine learning technique, used to predict the value of the dependent variable for new, unseen data.
- Regression is a statistical approach used to analyze the relationship between a dependent (target) variable and one or more independent (predictor) variables.
- More specifically, Regression analysis helps us to understand how the value of the dependent variable is changing corresponding to an independent variable when other independent variables are held fixed.
- It seeks to find the best-fitting model, which can be utilized to make predictions or draw conclusions.
- It predicts continuous/real values such as temperature, weight, age, salary, price, etc.
- In Regression, we plot a graph between the variables which best fits the given datapoints, using this plot, the machine learning model can make predictions about the data. In simple words,
- *"Regression shows a line or curve that passes through all the datapoints on target-predictor graph in such a way that the vertical distance between the datapoints and the regression line is minimum."*
- The distance between datapoints and line tells whether a model has captured a strong relationship or not.
- Some examples of regression can be as:
  - ✓ Prediction of rain using temperature and other factors
  - ✓ Determining Market trends
  - ✓ Prediction of road accidents due to rash driving.

Terminologies Related to Regression Analysis:

- ***Response Variable:*** The primary factor to predict or understand in regression, also known as the dependent variable or target variable.
- ***Predictor Variable:*** Factors which affect the response variable used to predict its values; also called independent variables.
- ***Outliers:*** Observations with significantly low or high values compared to others, potentially impacting results and best avoided.
- ***Multicollinearity:*** If the independent variables are highly correlated with each other than other variables, then such condition is called Multicollinearity. It should not be present in the dataset, because it creates problem while ranking the most affecting variable.
- ***Underfitting and Overfitting:*** Overfitting occurs when an algorithm performs well on training but poorly on testing, while underfitting indicates poor performance on both datasets.

Why do we use Regression Analysis?

- Regression estimates the relationship between the target and the independent variable.
- It is used to find the trends in data.
- It helps to predict real/continuous values.
- By performing the regression, we can confidently determine the *most important factor, the least important factor, and how each factor is affecting the other factors*.

Regression Types:

There are two main types of regression:

Simple Regression

- Simple linear regression should be used when there is only a single independent variable.
- Used to predict a continuous dependent variable based on a single independent variable.

Multiple Regression

- Multiple linear regression should be used when there are multiple independent variables.
- Used to predict a continuous dependent variable based on multiple independent variables.

NonLinear Regression

- Relationship between the dependent variable and independent variable(s) follows a nonlinear pattern.
- Provides flexibility in modeling a wide range of functional forms.

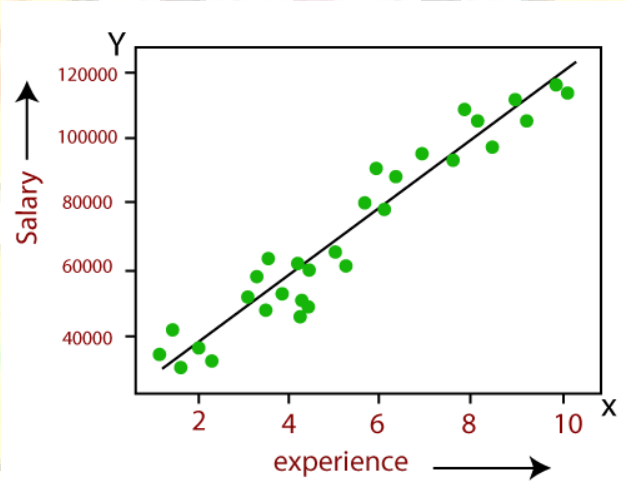
Regression Algorithms:

There are many different types of regression algorithms, but some of the most common include:

- ✓ Linear Regression
- ✓ Logistic Regression
- ✓ Polynomial Regression
- ✓ Support Vector Regression
- ✓ Decision Tree Regression
- ✓ Random Forest Regression
- ✓ Ridge Regression
- ✓ Lasso Regression

1. Linear Regression

- Linear regression is a statistical regression method which is used for predictive analysis.
- Linear regression shows the linear relationship between the independent variable (X-axis) and the dependent variable (Y-axis), hence called linear regression.
- If there is only one input variable (x), then such linear regression is called simple linear regression. And if there is more than one input variable, then such linear regression is called multiple linear regression.
- *Example:* predicting the salary of an employee on the basis of the year of experience.



- Below is the mathematical equation for Linear regression: **Y= aX+b**

  Where, Y = dependent variables (target variables),

  X= Independent variables (predictor variables),

  a and b are the linear coefficients
- Some popular applications of linear regression are:
  - ✓ Analyzing trends and sales estimates
  - ✓ Salary forecasting
  - ✓ Real estate prediction

- ✓ Arriving at ETAs in traffic.

2. ***Logistic Regression***

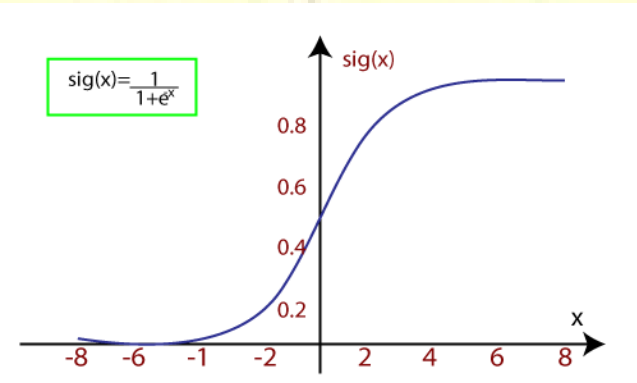
- Logistic regression is another supervised learning algorithm which is used to solve the classification problems.
- Logistic regression algorithm works with the categorical variable such as 0 or 1, Yes or No, True or False, Spam or not spam, etc.
- It is a predictive analysis algorithm which works on the concept of probability.
- Logistic regression uses sigmoid function or logistic function used to model the data which is a complex cost function. The function can be represented as:

$$f(x) = \frac{1}{1+e^{-x}}$$

Where, f(x)= Output between the 0 and 1 value.

x = input to the function

e = base of natural logarithm.

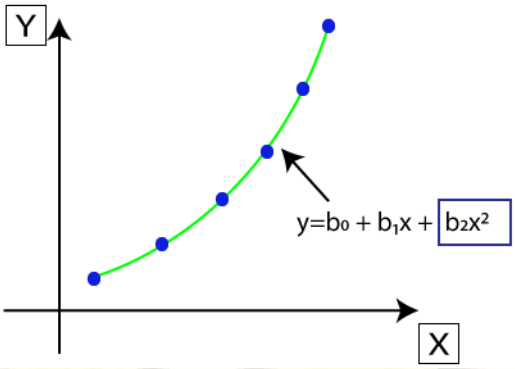


- It uses the concept of threshold levels, values above the threshold level are rounded up to 1, and values below the threshold level are rounded up to 0.
- There are three types of logistic regression:
  - ✓ Binary(0/1, pass/fail)
  - ✓ Multi(cats, dogs, lions)
  - ✓ Ordinal(low, medium, high)

3. ***Polynomial Regression:***

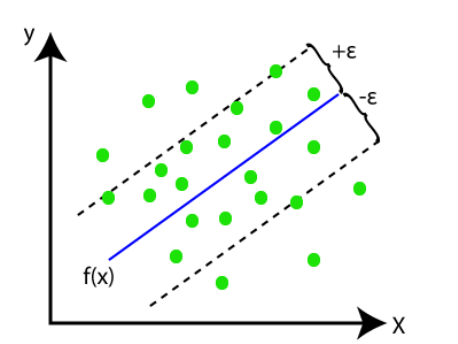
- Polynomial Regression is a type of regression which models the non-linear dataset using a linear model.
- It is similar to multiple linear regression, but it fits a non-linear curve between the value of x and corresponding conditional values of y.
- Suppose there is a dataset which consists of datapoints which are present in a non-linear fashion, so for such case, linear regression will not best fit to those datapoints. To cover such datapoints, we need Polynomial regression.
- In Polynomial regression, the original features are transformed into polynomial features of given degree and then modeled using a linear model. This means the datapoints are best fitted using a polynomial line.
- The equation for polynomial regression also derived from linear regression equation that means Linear regression equation $Y= b_0+ b_1x$, is transformed into Polynomial regression equation $Y= b_0+b_1x+ b_2x^2+ b_3x^3+.....+ b_nx^n$.
- Here Y is the predicted/target output, b_0, b_1,... b_n are the regression coefficients. x is our independent/input variable.

- The model is still linear as the coefficients are still linear with quadratic



4. ***Support Vector Regression***
   - Support Vector Machine is a supervised learning algorithm which can be used for regression as well as classification problems.
   - Support Vector Regression is a regression algorithm which works for continuous variables.
   - In Support Vector Regression we use:
     - ✓ ***Kernel:*** It is a function used to map a lower-dimensional data into higher dimensional data.
     - ✓ ***Hyperplane:*** In general SVM, it is a separation line between two classes, but in SVR, it is a line which helps to predict the continuous variables and cover most of the datapoints.
     - ✓ ***Boundary line:*** Boundary lines are the two lines apart from hyperplane, which creates a margin for datapoints.
     - ✓ ***Support vectors:*** Support vectors are the datapoints which are nearest to the hyperplane and opposite class.

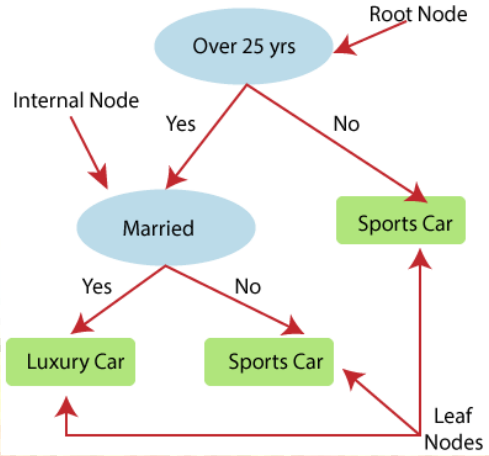
In SVR, we always try to determine a hyperplane with a maximum margin, so that maximum number of datapoints are covered in that margin. *The main goal of SVR is to consider the maximum datapoints within the boundary lines and the hyperplane (best-fit line) must contain a maximum number of datapoints.*



Here, the blue line is called hyperplane, and the other two lines are known as boundary lines.

5. ***Decision Tree Regression:***
   - Decision Tree is a supervised learning algorithm which can be used for solving both classification and regression problems.
   - It can solve problems for both categorical and numerical data
   - Decision Tree regression builds a tree-like structure in which each internal node represents the "test" for an attribute, each branch represent the result of the test, and each leaf node represents the final decision or result.
   - A decision tree is constructed starting from the root node/parent node (dataset), which splits into left and right child nodes (subsets of dataset). These child nodes are further divided into their

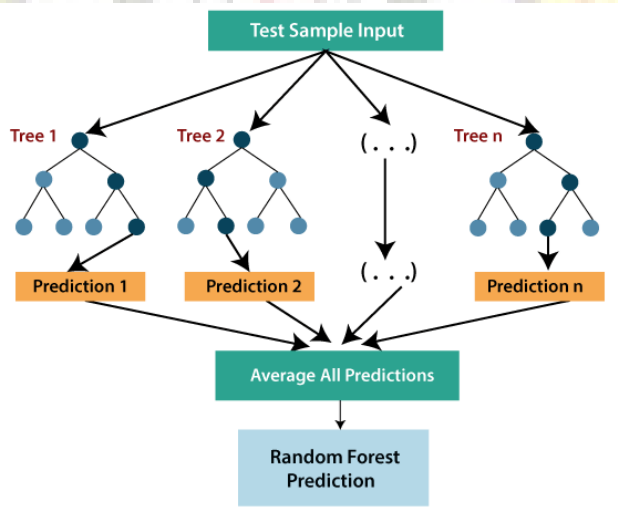
children node, and themselves become the parent node of those nodes.



6. ***Random Forest Regression:***
   - Random forest is one of the most powerful supervised learning algorithms which is capable of performing regression as well as classification tasks.
   - The Random Forest regression is an ensemble learning method which combines multiple decision trees and predicts the final output based on the average of each tree output. The combined decision trees are called as base models, and it can be represented more formally as:

$$g(x)= f_0(x)+ f_1(x)+ f_2(x)+....$$

   - Random forest uses ***Bagging or Bootstrap Aggregation*** technique of ensemble learning in which aggregated decision tree runs in parallel and do not interact with each other.
   - With the help of Random Forest regression, we can prevent Overfitting in the model by creating random subsets of the dataset.



7. ***Ridge Regression:***
   - Ridge regression is one of the most robust versions of linear regression in which a small amount of bias is introduced so that we can get better long term predictions.
   - The amount of bias added to the model is known as ***Ridge Regression penalty***. We can compute this penalty term by multiplying with the lambda to the squared weight of each individual features.
   - The equation for ridge regression will be:

$$L(x,y) = Min(\sum_{i=1}^{n}(y_i - w_i x_i)^2 + \lambda \sum_{i=1}^{n}(w_i)^2)$$

   - A general linear or polynomial regression will fail if there is high collinearity between the

independent variables, so to solve such problems, Ridge regression can be used.

- Ridge regression is a regularization technique, which is used to reduce the complexity of the model. It is also called as L2 regularization.
- It helps to solve the problems if we have more parameters than samples.

8. ***Lasso Regression:***

- Lasso regression is another regularization technique to reduce the complexity of the model.
- It is similar to the Ridge Regression except that penalty term contains only the absolute weights instead of a square of weights.
- Since it takes absolute values, hence, it can shrink the slope to 0, whereas Ridge Regression can only shrink it near to 0.
- It is also called as L1 regularization. The equation for Lasso regression will be:

$$L(x, y) = Min(\sum_{i=1}^{n} (y_i - w_i x_i)^2 + \lambda \sum_{i=1}^{n} |w_i|)$$

Assessing performance of Regression - Error measures:

- Evaluating performance of a regression model requires a different approach and different metrics than are used to evaluate classification models.
- Regression models estimate continuous values; therefore, regression performance metrics quantify how close model predictions are to actual (true) values.
- The following are some commonly used performance metrics for regression:
  - ✓ Mean Absolute Error (MAE)
  - ✓ Mean Squared Error (MSE)
  - ✓ Root Mean Squared Error (RMSE)
  - ✓ Huber Loss
  - ✓ R^2 – Score
  - ✓ Root Mean Square Logarithmic Error

1. **Mean Absolute Error (MAE):**

- Mean Absolute Error calculates the average difference between the calculated values and actual values.
- It is also known as scale-dependent accuracy used to predict the accuracy of the machine learning model.
- The less the value of MAE the better the performance of your model.

$$MAE = \frac{1}{n}\sum_{i=1}^{n} | Actual_i - Predicted_i |$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n} | y_i - \hat{y}_i |$$

Where, n is the total number of evaluated samples, y_i is the ith observed (actual) output, and $\hat{y}_i$ is the ith predicted output.

Actual	Predicted	
2.5	2.1	Now, using the formula we get **MAE = 0.22**
3.7	3.9	
1.8	1.7	
4.0	3.8	
5.2	5.0	

2. Mean Squared Error (MSE):

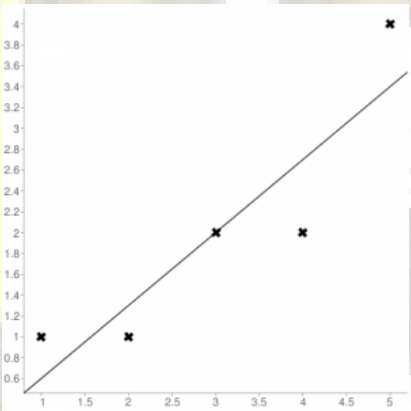
- The Mean Squared Error (MSE) or Mean Squared Deviation (MSD) of an estimator measures the average of error squares i.e. the average squared difference between the estimated values and true value.
- It is a risk function, corresponding to the expected value of the squared error loss. It is always non – negative and values close to zero are better.
- The MSE is the second moment of the error (about the origin) and thus incorporates both the variance of the estimator and its bias.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}(Actual_i - Predicted_i)^2$$

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Example: Consider the given data points: (1,1), (2,1), (3,2), (4,2), (5,4)

Regression line equation: Y = 0.7X – 0.1



X	Y	$\hat{Y}_i$	
1	1	0.6	Now, using formula we can get **MSE = 0.21606**
2	1	1.29	
3	2	1.99	
4	2	2.69	
5	4	3.4	

Actual	Predicted	
2.5	2.1	Now, using the formula we get **MSE = 0.058**
3.7	3.9	
1.8	1.7	
4.0	3.8	
5.2	5.0	

3. Root Mean Squared Error (RMSE)

- RMSE is the square root of MSE. Root Mean Square Error is the measure of how well a regression line fits the data points.
- RMSE is a quadratic measure of the error between predicted and observed values.
- It is similar to MAE as a way to measure the magnitude of model error, but because RMSE averages the square of errors, it provides a higher weight to large magnitude errors.

- RMSE is a commonly used metric in business problems where higher magnitude errors have a higher consequence – like predicting item sales prices, where high-priced items matter more for bottom-line business goals.
- However, this also may result in over-sensitivity to outliers.

$$RMSE = \sqrt{MSE}$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Actual_i - Predicted_i)^2}$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Actual	Predicted	
2.5	2.1	Now, using the formula we get **RMSE = 0.24083**
3.7	3.9	
1.8	1.7	
4.0	3.8	
5.2	5.0	

4. Huber Loss

- The mean squared error (MSE) or squared error gives too much importance to outliers and Mean Average error (MAE) gives equal weightage to all points.
- Huber loss combines MSE and MAE to give best of both worlds - it is quadratic (MSE) when the error is small else MAE for linear.
- For a loss value less than delta we use MSE and for loss value greater then delta we use MAE.
- The delta value is a hyperparameter and it determines the threshold.

$$L_\delta = \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2, & if\ |y_i - \hat{y}_i| < \delta \\ \delta\left((y_i - \hat{y}_i) - \frac{1}{2}\delta\right), & otherwise \end{cases}$$

Where, L represents the Huber Loss function

δ is the delta parameter, which determines the threshold for switching between the quadratic and linear components of the loss function.

5. R-squared (R^2) Score

- R-squared (R^2) score, also referred to as the coefficient of determination is a statistical method that determines the goodness of fit.
- It is the amount of the variation in the output dependent attribute which is predictable from the input independent variable(s).
- It measures the strength of the relationship between the dependent and independent variables on a scale of 0-100%.
- The high value of R-square determines the less difference between the predicted values and actual values and hence represents a good model.
- It is used to check how well-observed results are reproduced by the model, depending on the ratio of total deviation of results described by the model.
- It ranges from 0 to 1. A higher R^2 value (closer to 1) indicates a better fit.
- An R^2 of 0.8 means that 80% of the variance in the target variable is explained by the model.

$$R^2 = 1 - \frac{Sum\ of\ Squared\ Residuals\ (SSR)}{Total\ Sum\ of\ Squares\ (SST)}$$

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y}_i)^2}$$

Where, SSR/RSS is the sum of the squared difference between the actual and predicted value.

SST is the sum of the squared difference between the actual and average value i.e., the total variance in the dependent variable.

Actual	Predicted	
2.5	2.1	Now, using the formula we get **RMSE = 0.958877**
3.7	3.9	
1.8	1.7	
4.0	3.8	
5.2	5.0	

Overfitting

In the real world, the dataset present will never be clean and perfect. It means each dataset contains impurities, noisy data, outliers, missing data, or imbalanced data. Due to these impurities, different problems occur that affect the accuracy and the performance of the model. One of such problems is Overfitting in Machine Learning.

"A statistical model is said to be overfitted if it can't generalize well with unseen data."

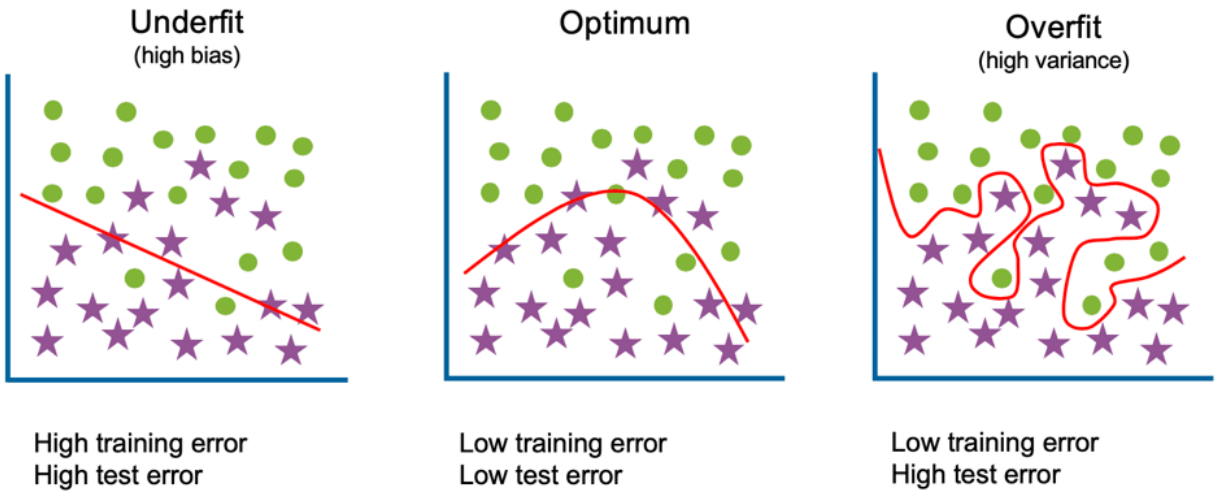
Before understanding overfitting, we need to know some basic terms, which are:

- ✓ ***Signal:*** It refers to the true underlying pattern of the data that helps the machine learning model to learn from the data.
- ✓ ***Noise:*** Noise is meaningless or irrelevant data present in the dataset that affects the performance of the model if it is not removed.
- ✓ ***Bias:*** Bias is a prediction error that is introduced in the model due to oversimplifying the machine learning algorithms. Or it is the difference between the predicted values and the actual values. When a model has poor performance both on the training and testing data means high bias because of the simple model, indicating ***underfitting***.
- ✓ ***Variance:*** Variance, on the other hand, is the error due to the model's sensitivity to fluctuations in the training data. If the machine learning model performs well with the training dataset, but does not perform well with the test dataset, then variance occurs. High variance occurs when a model learns the training data's noise and random fluctuations rather than the underlying pattern. As a result, the model performs well on the training data but poorly on the testing data, indicating overfitting.
- ✓ ***Generalization:*** It shows how well a model is trained to predict unseen data.

What is Overfitting?

- Overfitting is a problem where the evaluation of machine learning algorithms on training data is different from unseen data.
- Overfitting & underfitting are the two main errors/problems in the machine learning model, which cause poor performance in Machine Learning.
- Overfitting occurs when the model fits more data than required, and it tries to capture each and every datapoint fed to it. Hence it starts capturing noise and inaccurate data from the dataset, which degrades the performance of the model.

- An overfitted model doesn't perform accurately with the test/unseen dataset and can't generalize well.
- An overfitted model is said to have low bias and high variance.



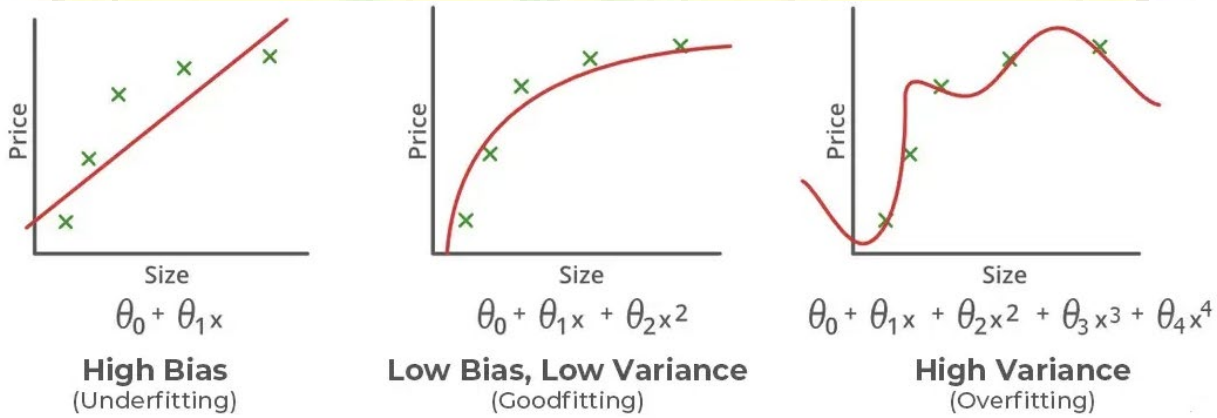
How to detect Overfitting?

- Overfitting in the model can only be detected once you test the data. To detect the issue, we can perform Train/test split.
- In the train-test split of the dataset, we can divide our dataset into random test and training datasets. We train the model with a training dataset which is about 80% of the total dataset. After training the model, we test it with the test dataset, which is 20 % of the total dataset.

Now, if the model performs well with the training dataset but not with the test dataset, then it is likely to have an overfitting issue.

For example, if the model shows 85% accuracy with training data and 50% accuracy with the test dataset, it means the model is not performing well.

Catalysts for Overfitting



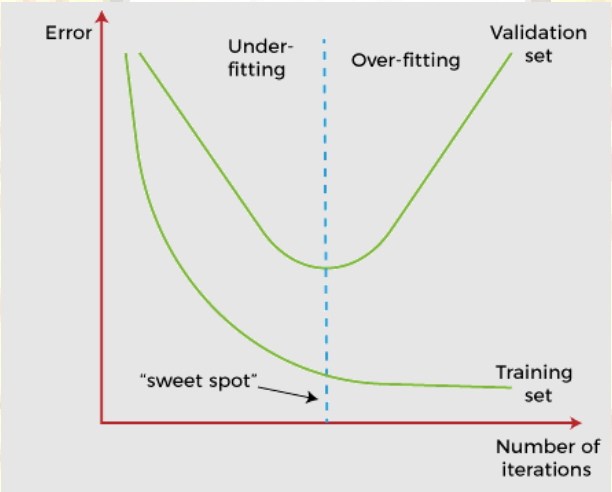
Reasons for Overfitting:

- Training data contains noise or garbage values in it.
- The model has high variance and low bias.
- The model is too complex.
- The size of the training dataset is insufficient.

Techniques to Reduce Overfitting:

Early stopping during the training phase:

- In this technique, the training is paused before the model starts learning the noise within the model.
- Stopping the training process before the model starts capturing noise from the data is known as early stopping.
- However, this technique may lead to the underfitting problem if training is paused too early. So, it is very important to find that "sweet spot" between underfitting and overfitting.
- There are two other methods by which we can get a good point for our model, which are the *resampling method* to estimate model accuracy and *validation dataset*.



Increase training data:

- Increasing the training set by including more data can enhance the accuracy of the model, as it provides more chances to discover the relationship between input and output variables.
- It may not always work to prevent overfitting, but this way helps the algorithm to detect the signal better to minimize the errors.
- When a model is fed with more training data, it will be unable to overfit all the samples of data and forced to generalize well.
- But in some cases, the additional data may add more noise to the model; hence we need to be sure that data is clean and free from in-consistencies before feeding it to the model.

Feature Selection:

- While building the Machine Learning model, we have a number of parameters or features that are used to predict the outcome.
- However, sometimes some of these features are redundant or less important for the prediction, and for this feature selection process is applied.
- In the feature selection process, we identify the most important features within training data, and other features are removed.
- Further, this process helps to simplify the model and reduces noise from the data.
- Some algorithms have the auto-feature selection, and if not, then we can manually perform this process.

Cross-Validation:

- Cross-validation is one of the powerful techniques to prevent overfitting.
- In the general k-fold cross-validation technique, we divided the dataset into k-equal-sized subsets of data; these subsets are known as folds.

Reduce model complexity:

- Using overly complex models with too many parameters can lead to overfitting.
- Choose simpler models or apply regularization techniques (such as L1 or L2 regularization) to prevent excessive complexity.

Data Augmentation:

- Data Augmentation is a data analysis technique, which is an alternative to adding more data to prevent overfitting. In this technique, instead of adding more training data, slightly modified copies of already existing data are added to the dataset.
- The data augmentation technique makes it possible to appear data sample slightly different every time it is processed by the model. Hence each data set appears unique to the model and prevents overfitting.

Regularization:

- Regularization is the most popular technique to prevent overfitting. It is a group of methods that forces the learning algorithms to make a model simpler.
- Applying the regularization technique may slightly increase the bias but slightly reduces the variance.
- In this technique, we modify the objective function by adding the penalizing term, which has a higher value with a more complex model.
- The two commonly used regularization techniques are L1 (Lasso) Regularization and L2 (Ridge) Regularization.

Ensemble Methods:

- In ensemble methods, prediction from different machine learning models is combined to identify the most popular result.
- The most commonly used ensemble methods are *Bagging* and *Boosting*.
- In bagging, individual data points can be selected more than once. After the collection of several sample datasets, these models are trained independently, and depending on the type of task-i.e., regression or classification-the average of those predictions is used to predict a more accurate result. Moreover, bagging reduces the chances of overfitting in complex models.
- In boosting, a large number of weak learners arranged in a sequence are trained in such a way that each learner in the sequence learns from the mistakes of the learner before it. It combines all the weak learners to come out with one strong learner. In addition, it improves the predictive flexibility of simple models.

Reasons for Underfitting:

- The model is too simple, So it may be not capable to represent the complexities in the data.
- The input features which is used to train the model is not the adequate representations of underlying factors influencing the target variable.
- The size of the training dataset used is not enough.
- Excessive regularization are used to prevent the overfitting, which constraint the model to capture the data well.
- Features are not scaled.

Techniques to Reduce Underfitting:

- Increase model complexity.

- Increase the number of features, performing feature engineering.
- Remove noise from the data.
- Increase the number of epochs or increase the duration of training to get better results.

Case study of Polynomial Regression

Polynomial Regression

- Polynomial Regression is a form of linear regression in which the relationship between the independent variable x and dependent variable y is modelled as an nth-degree polynomial.
- The Polynomial Regression equation is given below:

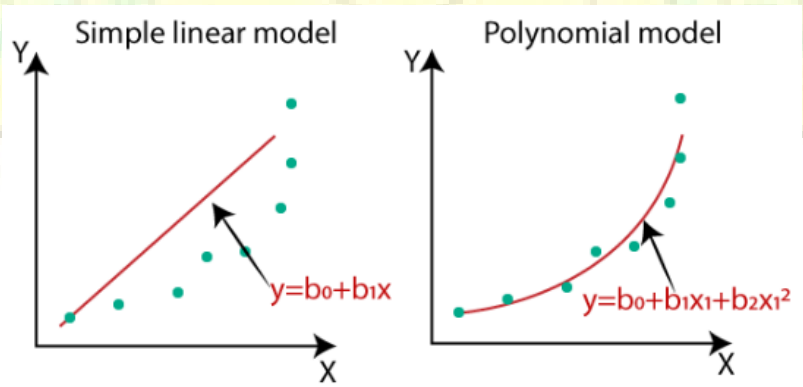
$$y= b_0+b_1x_1+ b_2x_1^2+ b_3x_1^3+..... b_nx_1^n$$

- Polynomial regression fits a nonlinear relationship between the value of x and the corresponding conditional mean of y, denoted E(y | x). In this article, we'll go in-depth about polynomial regression.
- There are some relationships that a researcher will hypothesize is ***curvilinear***. Clearly, such types of cases will include a polynomial term.
- It is also called the special case of Multiple Linear Regression in Machine Learning. Because we add some polynomial terms to the Multiple Linear regression equation to convert it into Polynomial Regression.
- It is a linear model with some modification in order to increase the accuracy.
- The dataset used in Polynomial regression for training is of non-linear nature.
- It makes use of a linear regression model to fit the complicated and non-linear functions and datasets.
- Hence, "In Polynomial regression, the original features are converted into Polynomial features of required degree (2, 3, .., n) and then modeled using a linear model."

Why Polynomial Regression?

The need of Polynomial Regression in Machine Learning is as follows:

- If we apply a linear model on a ***linear dataset***, then it provides us a good result as we have seen in ***Simple Linear Regression***, but if we apply the same model without any modification on a ***non-linear dataset***, then it will produce a drastic output. Due to which loss function will increase, the error rate will be high, and accuracy will be decreased.
- So for such cases, ***where data points are arranged in a non-linear fashion, we need the Polynomial Regression model***. We can understand it in a better way using the below comparison diagram of the linear dataset and non-linear dataset.



- A Polynomial Regression algorithm is also called Polynomial Linear Regression because it does not depend on the variables, instead, it depends on the coefficients, which are arranged in a linear fashion.

Equation of the Polynomial Regression Model:

Simple Linear Regression equation : $y = b_0+b_1x$

Multiple Linear Regression equation : $y= b_0+b_1x+ b_2x_2+ b_3x_3+....+ b_nx_n$

Polynomial Regression equation : $y= b_0+b_1x + b_2x^2+ b_3x^3+....+ b_nx^n$

Polynomial Regression Real-Life Example

- **Problem Description:** There is a Human Resource company, which is going to hire a new candidate. The candidate has told his previous salary 160K per annum, and the HR have to check whether he is telling the truth or bluff. So to identify this, they only have a dataset of his previous company in which the salaries of the top 10 positions are mentioned with their levels. By checking the dataset available, we have found that there is a **non-linear relationship between the Position levels and the salaries**. Our goal is to build a **Bluffing detector regression** model, so HR can hire an honest candidate.

Position	Level(X-variable)	Salary(Y-Variable)
Business Analyst	1	45000
Junior Consultant	2	50000
Senior Consultant	3	60000
Manager	4	80000
Country Manager	5	110000
Region Manager	6	150000
Partner	7	200000
Senior Partner	8	300000
C-level	9	500000
CEO	10	1000000

Steps for Polynomial Regression:

The main steps involved in Polynomial Regression are given below:

1. Data Pre-processing
2. Build a Linear Regression model and fit it to the dataset
3. Build a Polynomial Regression model and fit it to the dataset
4. Visualize the result for Linear Regression
5. Visualize the result for Polynomial Regression model.
6. Predicting the output.

1. Data Pre-processing Step:

- The data pre-processing step will remain the same as in previous regression models, except for some changes. In the Polynomial Regression model, we will not use feature scaling, and also we will not split our dataset into training and test set. It has two reasons:
- The dataset contains very less information which is not suitable to divide it into a test and training set, else our model will not be able to find the correlations between the salaries and levels.
- In this model, we want very accurate predictions for salary, so the model should have enough information.

The python script for pre-processing step is given below:

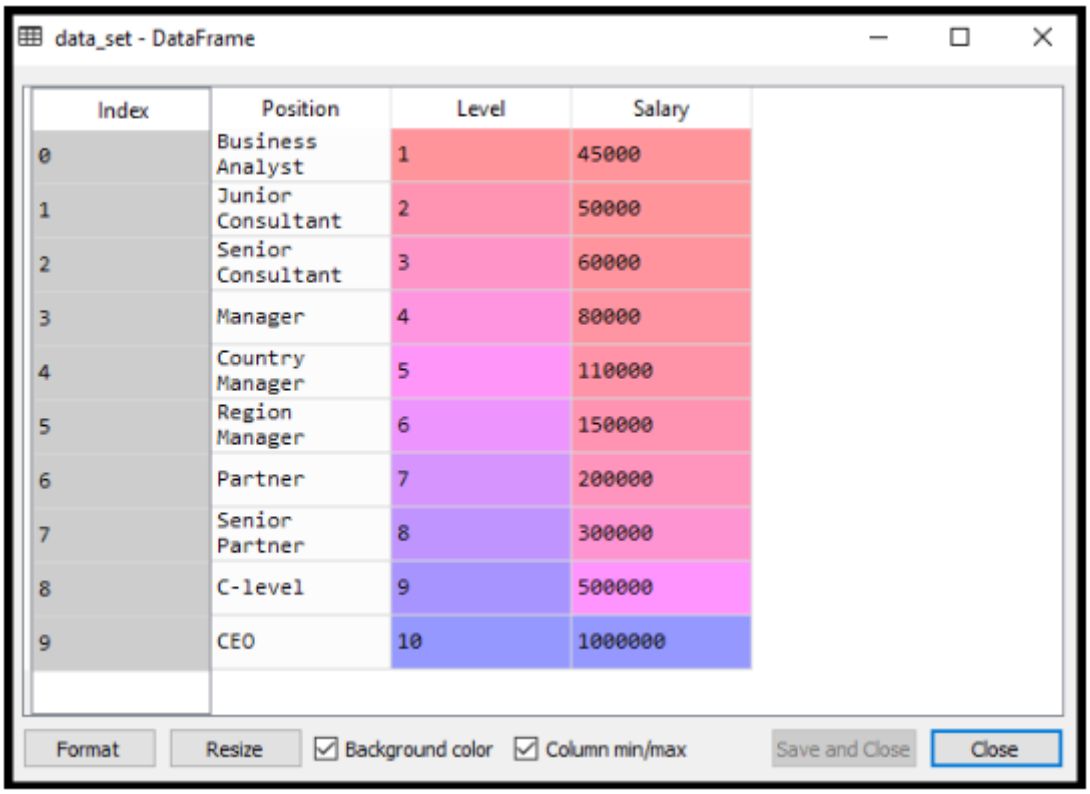
```
import numpy as nm
```

```
import matplotlib.pyplot as mtp
import pandas as pd
data_set= pd.read_csv('Position_Salaries.csv')        # importing datasets
# Extracting Independent and dependent Variable
x= data_set.iloc[:, 1:2].values
y= data_set.iloc[:, 2].values
```

Explanation:

- In the above lines of code, we have imported the important Python libraries to import dataset and operate on it.
- Next, we have imported the dataset 'Position_Salaries.csv', which contains three columns (Position, Levels, and Salary), but we will consider only two columns (Salary and Levels).
- After that, we have extracted the dependent(Y) and independent variable(X) from the dataset. For x-variable, we have taken parameters as [:, 1:2], because we want 1 index(levels), and included :2 to make it as a matrix.

Output:-



data_set - DataFrame

Index	Position	Level	Salary
0	Business Analyst	1	45000
1	Junior Consultant	2	50000
2	Senior Consultant	3	60000
3	Manager	4	80000
4	Country Manager	5	110000
5	Region Manager	6	150000
6	Partner	7	200000
7	Senior Partner	8	300000
8	C-level	9	500000
9	CEO	10	1000000

Format | Resize | Background color | Column min/max | Save and Close | Close

- As we can see in the above output, there are three columns present (Positions, Levels, and Salaries). But we are only considering two columns because Positions are equivalent to the levels or may be seen as the encoded form of Positions.
- Here we will predict the output for level 6.5 because the candidate has 4+ years' experience as a regional manager, so he must be somewhere between levels 7 and 6.

2. Building the Linear regression model:

- Now, we will build and fit the Linear regression model to the dataset. In building polynomial regression, we will take the Linear regression model as reference and compare both the results.
- We create a Simple Linear model and fitted it to the dataset variables (x and y).

```
# Fitting the Linear Regression to the dataset
from sklearn.linear_model import LinearRegression
lin_regs= LinearRegression()
```

```
lin_regs.fit(x,y)
```

In the above python script, we have created the Simple Linear model using lin_regs object of LinearRegression class and fitted it to the dataset variables (x and y).

Output:

Out[5]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)

3. Building the Polynomial regression model:

- Now we will build the Polynomial Regression model, but it will be a little different from the Simple Linear model.
- Because here we will use PolynomialFeatures class of preprocessing library. We are using this class to add some extra features to our dataset.

```
# Fitting the Polynomial regression to the dataset
from sklearn.preprocessing import PolynomialFeatures
poly_regs= PolynomialFeatures(degree= 2)
x_poly= poly_regs.fit_transform(x)
lin_reg_2 =LinearRegression()
lin_reg_2.fit(x_poly, y)
```

- In the above lines of code, we have used poly_regs.fit_transform(x), because first we are converting our feature matrix into polynomial feature matrix, and then fitting it to the Polynomial regression model. The parameter value (degree= 2) depends on our choice. We can choose it according to our Polynomial features.
- After executing the code, we will get another matrix x_poly, which can be seen under the variable explorer option:

	0	1	2
0	1	1	1
1	1	2	4
2	1	3	9
3	1	4	16
4	1	5	25
5	1	6	36
6	1	7	49
7	1	8	64
8	1	9	81
9	1	10	100

- Next, we have used another LinearRegression object, namely lin_reg_2, to fit our x_poly vector to the linear model.

Output:-

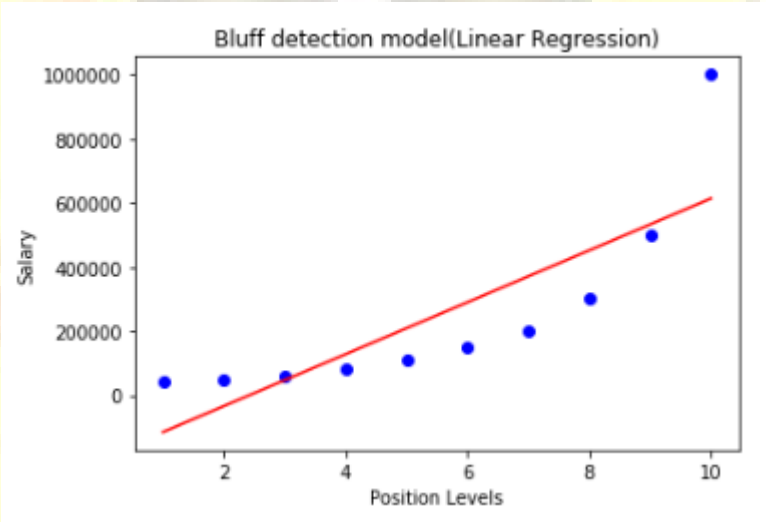
Out[11]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)

4. Visualizing the result for Linear regression:

- Now we will visualize the result for Linear regression model as we did in Simple Linear Regression. See the python script below:

```
#Visulaizing the result for Linear Regression model
mtp.scatter(x,y,color="blue")
mtp.plot(x,lin_regs.predict(x), color="red")
mtp.title("Bluff detection model(Linear Regression)")
mtp.xlabel("Position Levels")
mtp.ylabel("Salary")
mtp.show()
```

Output:-



- In the above output image, we can clearly see that the regression line is so far from the datasets. Predictions are in a red straight line, and blue points are actual values. If we consider this output to predict the value of CEO, it will give a salary of approx. 600000$, which is far away from the real value.
- So we need a curved model to fit the dataset other than a straight line.

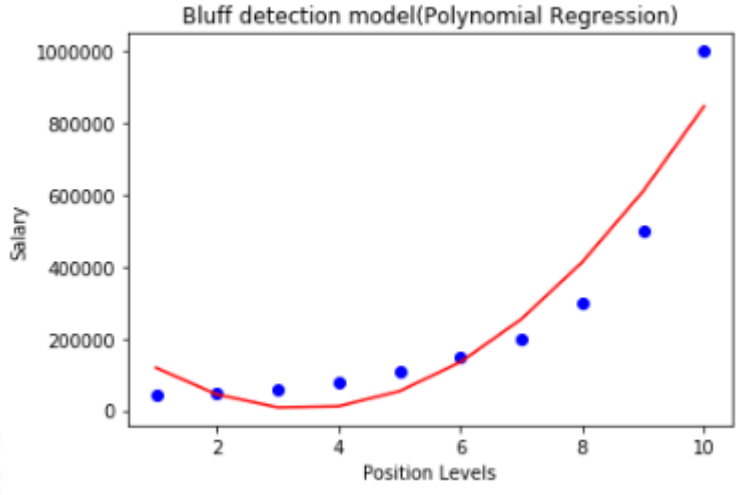
5. Visualizing the result for Polynomial Regression:

- Here we will visualize the result of Polynomial regression model, code for which is little different from the above model.
- See Python Script given below:

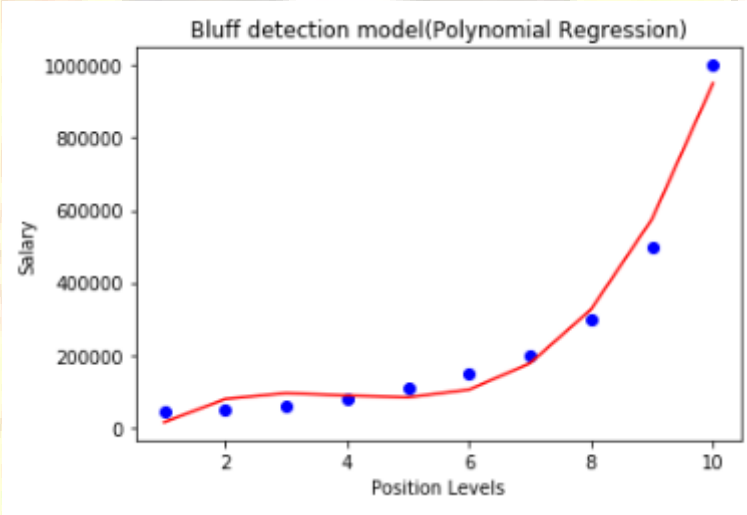
```
#Visulaizing the result for Polynomial Regression
mtp.scatter(x,y,color="blue")
mtp.plot(x, lin_reg_2.predict(poly_regs.fit_transform(x)), color="red")
mtp.title("Bluff detection model(Polynomial Regression)")
mtp.xlabel("Position Levels")
mtp.ylabel("Salary")
mtp.show()
```

- In the above code, we have taken lin_reg_2.predict(poly_regs.fit_transform(x), instead of x_poly, because we want a Linear regressor object to predict the polynomial features matrix.

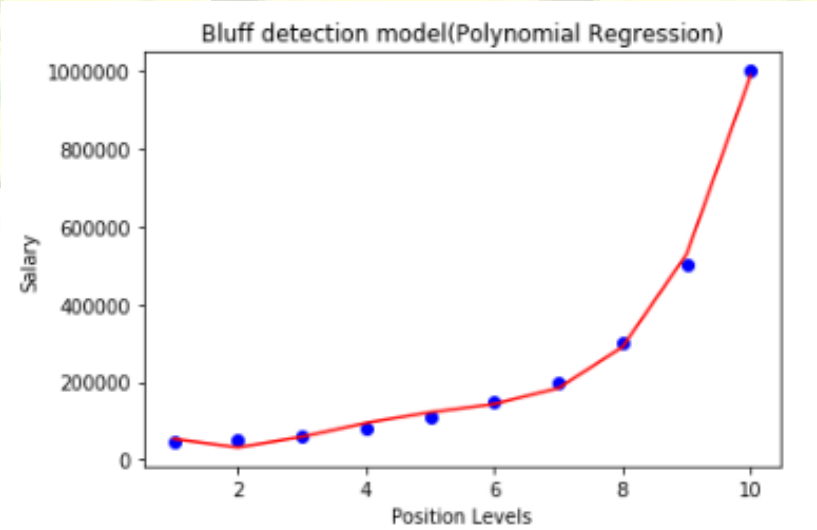
Output:-



- ***For degree= 3:*** If we change the degree=3, then we will give a more accurate plot, as shown in the below image.



- So, as we can see here in the above output image, the predicted salary for level 6.5 is near to 170K$-190k$, which seems that future employee is saying the truth about his salary.
- ***Degree= 4:*** Let's again change the degree to 4, and now will get the most accurate plot. Hence we can get more accurate results by increasing the degree of Polynomial.



6. Predicting the final result with the Linear Regression model:

Now, we will predict the final output using the Linear regression model to see whether an employee is saying truth or bluff. So, for this, we will use the predict() method and will pass the value 6.5. Below is the code for it:

```
lin_pred = lin_regs.predict([[6.5]])
print(lin_pred)
```

Output:-

[330378.78787879]

Predicting the final result with the Polynomial Regression model:

Now, we will predict the final output using the Polynomial Regression model to compare with Linear model. Below is the code for it:

```
poly_pred = lin_reg_2.predict(poly_regs.fit_transform([[6.5]]))
print(poly_pred)
```

Output:-

[158862.45265153]

As we can see, the predicted output for the Polynomial Regression is [158862.45265153], which is much closer to real value hence, we can say that future employee is saying true.